

Python Operators

- Operators are Symbols or words used to perform operations on value
- values used with operators are called operands
- This lesson covers arithmetic, assignment, comparison and logical operators
- we will also learn precedence, common mistake, and practice task.

Quick idea

a=10

b=3

print(a+b) # arithmetic

print(a > b) # comparison

print(a > 5) and b < 5) # logical

operator + operands $\rightarrow$ expression

Types of Operators Covered

Type	Main use	Examples
① Arithmetic	perform math	+, -, *, /, //, %, **
② Assignment	Store/update values	=, +=, -=, *=, /=
③ Comparison	Compare values	==, !=, >, <, >=, <=
④ Logical	combine condition	and, or, not

- Arithmetic usually returns a number.
- Assignment operators work with boolean condition.
- Logical operators update variables.
- Comparison usually returns True or False.

① Arithmetic Operators Overview

Operator	Name	Example	Result	Code
+	Addition	10+3	13	a = 10 b = 3 print(a+b)
-	Subtraction	10-3	7	print(a-b)
*	Multiplication	10*3	30	print(a*b)
/	True division	10/3	3.333	print(a/b)
//	Floor division	10//3	3	print(a//b)
%	Modulus	10%3	1	print(a%b)
**	Exponent	10**3	1000	print(a**b)

9) Addition, Subtraction, and Multiplication

- + adds numbers or joins strings.
- - Subtracts one number from another.
- * multiplies numbers. with strings, it repeats text.

Remember: "10" + "5" gives "105", not 15.

Number example	Strings example
x=20 y=5	First = "Python" Second = "Class"
print(x+y) # 25	print(First + " " + Second) → Python Class
print(x-y) # 15	print("Hi" * 3) # HiHiHi
print(x*y) # 100	

10) Division operators: /, //, and %

	Expression	meaning	Result
• / gives true division and returns a float	10/3	true division	3.333
• // gives floor division: quotient without decimal parts.	10//3	floor quotient	3
• % gives the remainder after division.	10%3	remainder	1
• These three are very important in problem solving	20%2	even check	0

Common use of %

```

number = 21
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
  
```

i) Exponent Operator: **

- ** means power or exponent.
- It is useful for Squares, Cubes, roots, and formulas
- It has higher priority than multiplication and division.

Expression

Result

$2^{**}3$

8

$5^{**}2$

25

$9^{**}0.5$

3.0

$10^{**}0$

1

Square: $x^{**}2$ Cube: $x^{**}3$

Codes example

Value = 27

Side = 5

Cube root = $\text{Value}^{**}(1/3)$

area = $\text{Side}^{**}2$

print(Cube root)

print(area) # 25

iv) Arithmetic Precedence

- Python follows operator precedence: Some operators run before others.
- Use parentheses when you want to make the order clear
- Parentheses make code easier for beginners to understand.

Priority

operators

meaning

1

()

Parentheses first

2

**

power

3

*, /, //, %

Multiply, divide, remainder

4

+, -

Addition / Subtraction

Example

print (2 + 3 * 4) # 14

print (2 + 3) * 4 # 20

print (2 ** 3 * 2) # 16

② Assignment Operators Overview

	operator	Long form	Short form
• Assignment operators store values in variables.	=	$x = 10$	$x = 10$
	+=	$x = x + 5$	$x += 5$
• = assigns a value	-=	$x = x - 5$	$x -= 5$
	*=	$x = x * 5$	$x *= 5$
• Compound assignment operators update an existing value in a shorter way.	/=	$x = x / 5$	$x /= 5$
	//=	$x = x // 5$	$x //= 5$
	%=	$x = x \% 5$	$x \% = 5$
	**=	$x = x ** 5$	$x ** = 5$

Assignment changes the stored value of a variable.

③ Assignment Operators: Step-by-Step

- Compound assignment saves time and keeps code clean.
- After division with /, the result becomes float.

code	Line	Value of Score
Score = 50	Score = 50	50
Score += 10 # Score = Score + 10	Score += 10	60
Score -= 5 # Score = Score - 5	Score -= 5	55
Score *= 2 # Score = Score * 2	Score *= 2	110
Score /= 5 # Score = Score / 5	Score /= 5	22.0
print (Score)		

② Comparison Operators Overview

Comparison operators compare two values	operator	meaning	example	Result
• They return a boolean result: True or False.	<code>==</code>	equal to	<code>5 == 5</code>	True
• They are commonly used inside if Statement	<code>!=</code>	not equal to	<code>5 != 3</code>	True
	<code>></code>	greater than	<code>5 > 3</code>	True
	<code><</code>	less than	<code>5 < 3</code>	False
	<code>>=</code>	greater or equal	<code>5 >= 5</code>	True
	<code><=</code>	less or equal	<code>4 <= 5</code>	True

Example

age = 18

`print (age >= 18) # True`

`print (age < 18) # False`

Pass/Fail example

mark = 72

`if marks >= 40%`

`print ("pass")`

else:

`print ("fail")`

Equality example

password = "abc 123"

`if password == "abc 123":`

`print ("Login Success")`

else:

`print ("wrong password")`

9) Comparison in if Statement

• Comparison operators help Python make decisions.

• This condition is checked first

• If the condition is True, the if block runs.

• If it's false, the else block can run.

90.1)	<code>=</code> vs <code>==</code>	Symbol	USE	example	meaning
•	<code>=</code> is assignment: it stores a value	<code>=</code>	assignment	<code>x = 10</code>	store 10 in x
•	<code>==</code> is comparison: it check whether two value are equal	<code>==</code>	comparison	<code>x == 10</code>	check if x is 10

• This is one of the most common beginner mistake.

Correct use

`x = 10 # assignment`

`print (x == 10) # Comparison: True`

`print (x == 5) # comparison: false`

Never write `if x = 10` in Python
 use `if x == 10`

④ Logical Operator Overview

- Logical operators combine or reverse condition.
- **and** : True only when both condition are True.
- **or** : True when at least one condition is True.
- **not** : reverses True to false and False to True.

Operator	meaning	Example
and	both condition must be True	age >= 18 and citizen == True
or	at least one condition is True	email == uses or phone == uses
not	reverse the result	not is_logged_in

Example

age = 20

has_id = True

print (age >= 18 and has_id) # True

9) Truth Table : and, or, not

A	B	A and B	A or B	A	not A
True	True	True	True	True	False
True	False	False	True	False	True
False	True	False	True		
False	False	False	False		

- Use and when all requirement must be Satisfied.
- Use or when at least one option is enough.
- Use not when you need the Opposite results.

Combining Operators

- A Single condition can use arithmetic, comparison, and Logical Operation together.
- First calculate arithmetic expression, then compare, then apply logical operators
- Use parentheses to make long condition readable.

Example
math = 80
Science = 70
English = 60
average = (math + Science + english) / 3
if average >= 60 and english >= 40:
print ("passed the good average")
else:
print ("needs improvement")

Part	Operator Types
math + Science + English	arithmetic
average >= 60	comparison
english >= 40	Comparison
Condition 1 and condition 2	logical

Operator Precedence Summary

	Priority	Operator group	Examples	Step idea
1	1	Parentheses	()	
	2	Exponent	**	result = $5+3>6$ and not false
	3	Arithmetic add/subtract	+, -	# $5+3$ first : 8
	4	Comparison	=, !=, >, <, >=, <=	# $8>6$ next : True
	5	Logical not	not	# not false : True
	6	Logical and	and	# True and True : True
	7	Logical or	or	
	8	Arithmetic multiply/divide	*, /, //, %	

Common Mistakes

Mistake	Problem	Correct idea
if $x = 10$:	$=$ is assignment	use if $x == 10$
"10" + 5	String + int error	convert: <code>int("10")</code>
$10 / 2$ gives 5	actually gives 5.0	use // if integer quotient needed
and used for alternative	and needs both True	use or for alternative
Forgetting parentheses	logic becomes unclear	write clear grouped